Chapter 3 Assignments

- Reading: Chapter 3 sections 1,2,3,4
  - (up to TCP, not including)
- Chapter 3 Practice
  - "R" Problems: R2 (think about letter-writing example in book/class), R3, R4, R5, R6, R7, R8, R9, R10, R11

1

1

Lecture this week and next:

- Intro to Transport Layer
- Internet Checksum
- Section 3.4: rdt 1.0, 2.0, 2.1
  Begin rdt protocols
- Internet structure
- GoBackN and Selective Repeat
- Then TCP

2

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

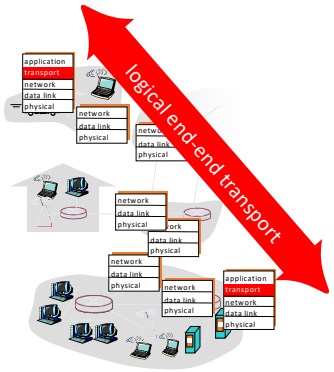
3.5 connection-oriented transport: TCP
- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

5

Transport Layer



Process to Process Communication

9

Goals

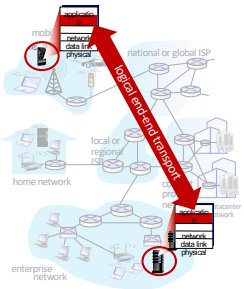
- Understand principles behind transport layer services:
  - multiplexing/demultiplexing
  - reliable data transfer
  - flow control
  - congestion
- Learn about the Internet transport layer protocols:
  - UDP
  - TCP

10

10

Transport services and protocols

- *Logical communication* between application processes running on different hosts
  - Appearance directly connected
- Two transport protocols available to Internet applications
  - TCP, UDP
- End systems:
  - TCP Sender: breaks application messages into *segments*, passes to network layer
  - Reconstruction at Receiver: reassembles segments into messages, passes up to Application Layer



11

Letter-writing example

12

Transport vs. network layer services and protocols

- Network layer: logical communication between *hosts*
- Transport layer: logical communication between *processes*
  - relies on, enhances, network layer services

household analogy:

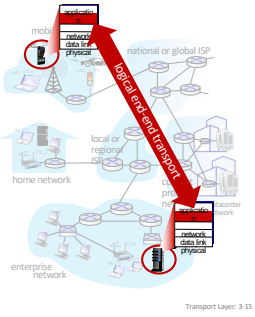
12 kids in Ann's house sending letters to 12 kids in Bill's house:

- hosts = houses
- processes = kids writing and reading letters
- app messages = letters in envelopes
- transport protocol = Ann and Bill who collect and distribute mail in the house
- network protocol = postal service

14

Two principal Internet transport protocols

- **TCP:** Transmission Control Protocol
  - reliable, in-order delivery
  - congestion control
  - flow control
  - connection setup
- **UDP:** User Datagram Protocol
  - unreliable, unordered delivery
  - no-frills extension of "best-effort" IP
- Services not available in the Internet:
  - delay guarantees
  - bandwidth guarantees



Transport Layer: 3-15

15

Transport Services and Protocols

UDP vs. TCP

16

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP
- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

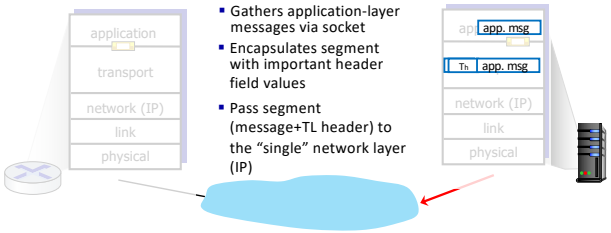
Transport Layer 3-18

18

Transport Layer Multiplexing

Transport Layer Sender:

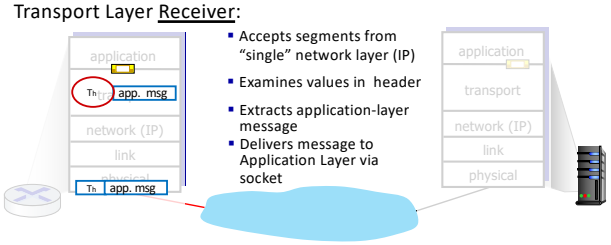
- Gathers application-layer messages via socket
- Encapsulates segment with important header field values
- Pass segment (message+TL header) to the "single" network layer (IP)



20

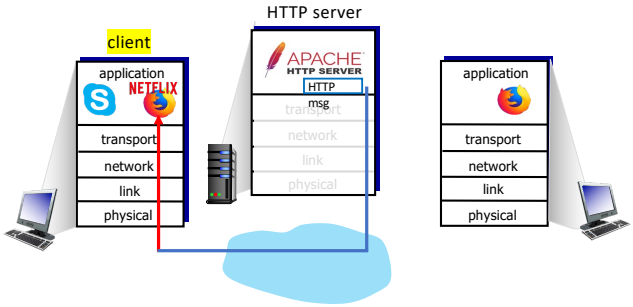
Transport Layer Demultiplexing

Transport Layer Receiver:



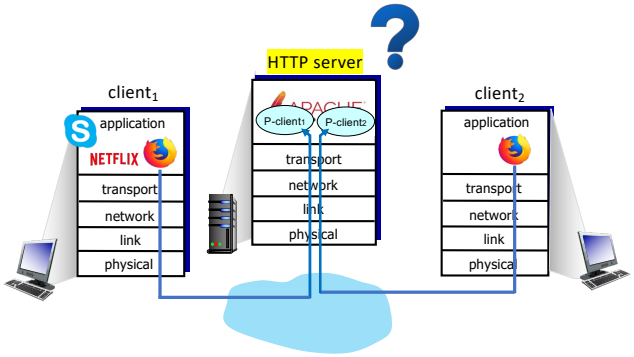
- Accepts segments from "single" network layer (IP)
- Examines values in header
- Extracts application-layer message
- Delivers message to Application Layer via socket

21



2 processes at Client host

22



2 Client hosts send to one server → how does HTTP server distinguish the incoming messages? → P1 or P2?
(Think back to the kids)

26

Multiplexing/demultiplexing

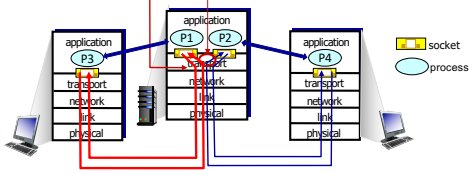
Extend network layer host-to-host delivery service to process-to-process delivery

Multiplexing at Sender:

Transport Layer pulls data from multiple sockets, adds header (port number, later used for de-multiplexing) and sends down to single Network Layer

Demultiplexing at Receiver:

Single Network layer delivers segment to Transport Layer. TL uses header info (port number)to deliver received segments to correct socket



27

Multiplexing/Demultiplexing

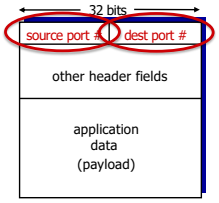
28

Note: Multiplexing homework problem is in Ch 2 problem set!

Ports are key to multiplexing/demultiplexing

Host receives IP datagram:

- Network Layer:
  - IP Header: src, dst IP addresses
  - Payload: Transport Layer Segment
- Transport Layer:
  - TL header: source and destination port number
  - Payload: App Layer data
  - Clients: Ephemeral ports assigned to client processes
  - Servers: bind to well-known ports

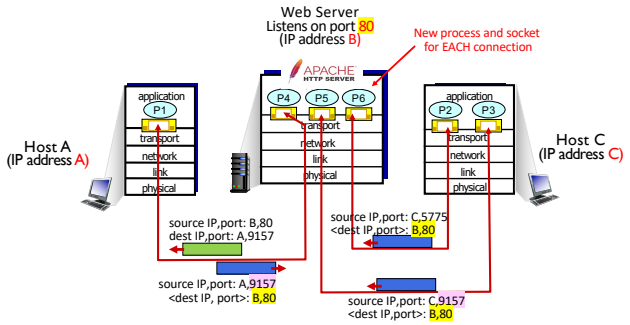


Transport Layer: TCP/UDP header

31

TCP demultiplexing: example

Study this!



- All three packets arriving at web server → <dest IP address, dest port> = <B,80>
  - Two packets have source port = 9157 (?)
  - Two packets have source IP = C

All must be demultiplexed to *different* sockets -- how does Server keep it straight?

33

Connection-oriented Multiplexing (TCP)

- Servers support many simultaneous TCP sockets:
  - Each socket associated with a different connecting client
- Each TCP socket/connection uniquely identified by 4-tuple:
  - source IP address
  - source port number
  - dest IP address
  - dest port number
    - Lots of sockets!
    - Overhead
- Each TCP connection can be:
  - non-persistent HTTP
  - Persistent
  - Pipeline
  - Parallel etc.
- Demux: *4-tuple values* used to direct segment to appropriate socket

34

Section 3.4 Reliable Data Transfer

- Implemented either at the link layer or the transport layer

General context:

- Transport Layer gives Application Layer impression that network is reliable
- Section 3.4:
  - Start with simple protocol
  - Build up as our assumptions about the network increase
- TCP – Section 3.5 (after midterm)

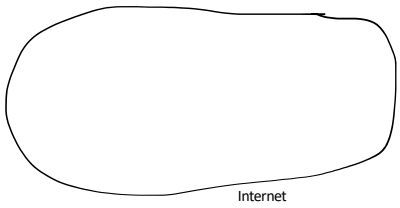
36

Reliable Data Transfer Abstraction

37

Sources of data loss

1.
2.



Error Recovery Techniques:

1. Detection
2. Recovery

39

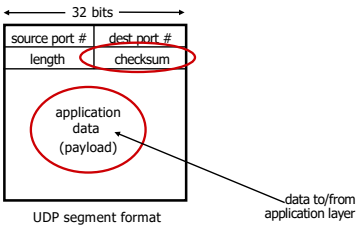
Internet Checksum

- Mechanisms to detect bit errors in transmitted packets are used throughout protocol stack
  - Transport Layer
  - Network Layer
  - Link Layer
- Internet Checksum
  - L4 and L3
- Repetition – consequence of modularity

41

For example, 16 bits in UDP Header

(TCP Header also has this field too)



42

Goal of Checksum

Goal: detect errors (*i.e.,* flipped bits) in transmitted segment

	1st number	2nd number	sum
Transmitted:	5	6	11
Received:	4	6	11

receiver-computed checksum ≠ sender-computed checksum (as received)

43

Internet checksum

Goal: Detect "errors" (e.g., flipped bits) in transmitted segment

Sender:
- Treat Payload as sequence of 16-bit integers
- Compute checksum:
  - addition of segment contents
  - take 1's complement
- Checksum value put into outgoing packet header

Receiver:
- Compute checksum of received packet
- Does computed checksum equal received checksum (field in header)?
  - NO - error detected
  - YES - no error **detected** *but possible errors nonetheless*

44

Example: Internet Checksum

Steps:
1. Add two 16-bit integers
2. If there's a carryout, add it back in
3. Flip the bits (1s complement)
4. The result in #3 is the checksum

Remember: a carryout from the most significant bit needs to be added to the result

```
                1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0
                1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1
wraparound    1 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1
Sum →           1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 0
Checksum →      0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1
(1s complement of the Sum)
```

45

Weak protection!

Example: add two 16-bit integers

```
                1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0   0 1
                1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1   1 0
wraparound    1 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1
sum             1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 0
checksum        0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1
```

Even though numbers have changed (bit flips), *no* change in checksum!

46

Checksum Overhead

- What is the checksum overhead?
  - --
  - --

47

Why reliable data transfer?

- Correct for losses in network
  - Transmission errors
  - Packet loss
- Goal: provide reliable data delivery service over an unreliable channel/network.

Requirements:

- All transmitted data is delivered to Receiver
- In order
- No corruption
- No duplicates

49

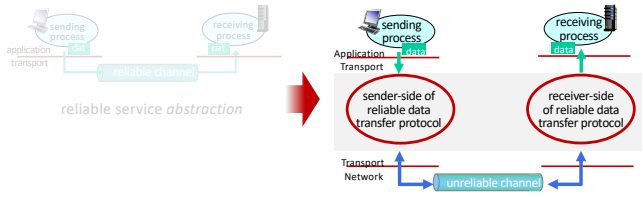
Principles of reliable data transfer 1

Transport Layer presents "Service Abstraction" to upper layer:

- Reliable network, black box
  - Sender writes → Receiver reads
    - what goes in, comes out
    - Bits delivered in order, no corruption and no duplicates

50

Principles of reliable data transfer (rdt) 2



reliable service *implementation*

Reliable Data Transfer protocol (rdt x) **implements** this service abstraction

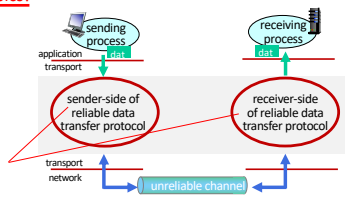
51

Principles of reliable data transfer 3

Top-10 list of important networking topics!

Complexity of reliable data transfer protocol will depend (strongly) on characteristics of unreliable channel *

IMPAIRMENTS introduced by channel



reliable service *implementation*

52

Impairments

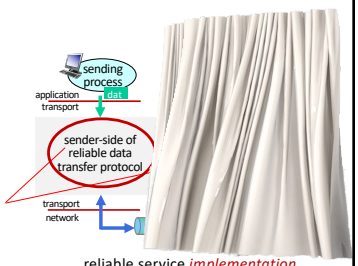
Characteristics of Unreliable Channel	How Service Guaranteed
-	-
-	-
-	-
-	-

53

Principles of reliable data transfer 4

Sender, Receiver do *not* know the "state" of each other, e.g., was a message received?

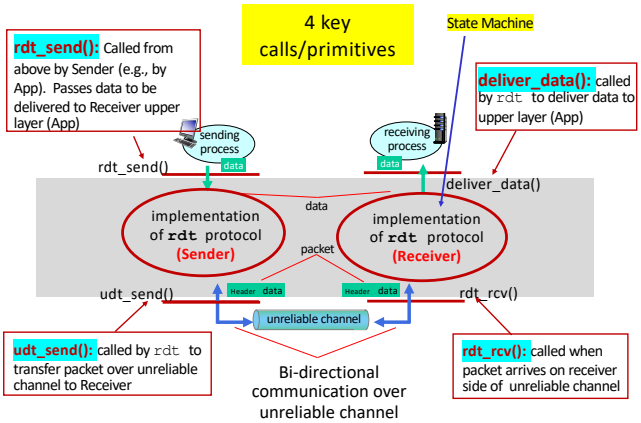
- Exchange messages to communicate state



reliable service *implementation*

54

rdt protocol (rdt): interfaces



55

Homework -- finish this before:

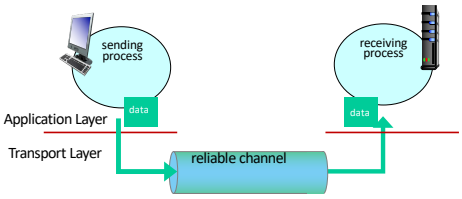
- Read Section 3.4
- Take notes on the 4 rdt primitaves
  - Name
  - Function
  - Data flow
- Small quiz on this

56

Finite State Machines (FSM) *

58

Reminder: What we are trying to achieve



reliable service *abstraction*

IMPAIRMENTS introduced by channel considered with each iteration of RDT protocol

Service Abstraction → reliable channel, black box
- What goes in is guaranteed to come out;
- in order;
- No corruption
- No duplicates.

60

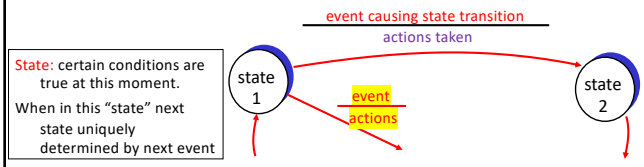
5 versions of rdt protocol based on channel characteristics

- **Rdt 1.0-** 100% reliable channel
- **Rdt 2.0** – Channel introduces bit errors →corrupted
  →
- **Rdt 2.1** - Channel has Bit errors in both directions(data packets as well as Receiver's feedback – ACKs and NAKs)
  →
- **Rdt 2.2** – get rid of NAKs
- **Rdt 3.0** - Channel with above errors *and* **PACKET LOSS**
  → What do we do about it?

61

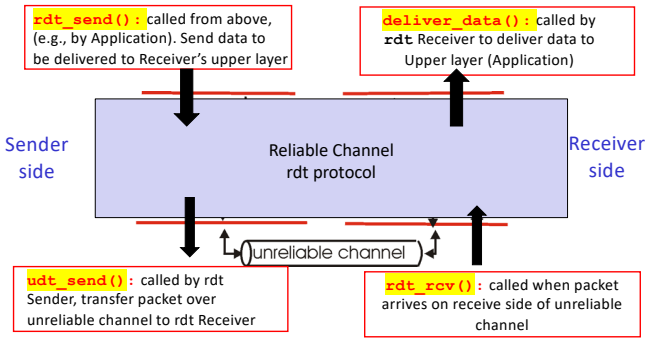
Getting started:

We will:
- Incrementally develop sender and receiver sides of reliable data transfer protocol (rdt)
- Consider only unidirectional transfer of **data**
  - control info flows in both directions!
- FSM specifies the actions of the Sender and Receiver



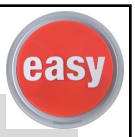
62

Reliable data transfer: 4 primitives

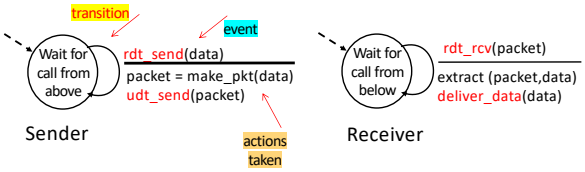


rdt1.0 *

Rdt1.0: Perfect Channel



- Assumption: Underlying channel perfectly reliable
  - no transmission/bit errors
  - no dropped packets
- Separate FSMs for Sender & Receiver:
  - Sender sends data into underlying perfect channel
  - Receiver reads data from underlying perfect channel



Note: no feedback from Receiver to Sender! Why?

Is Rdt1.0 assumption reasonable?

- Does a 100% reliable network exist?
- Good for showing use of primitives

Rdt2.0

Better Channel Assumptions:

- Bit errors on channel
- No packet loss

rdt2.0 Bit Errors on channel, no packet loss

- Channel Assumptions:
  - Bit errors on channel
  - No packet loss
- New mechanisms:
  - Checksum:
    - Sender includes for Error Detection
  - Receiver Feedback:
    - For each packet received, send control messages to Sender (ACK,NAK *)
  - Retransmissions:
    - If NAK received, Sender retransmits!

Stop and Wait
sender transmits one packet, waits for receiver's response

rdt2.0: 2 state FSM

event causing state transition
actions taken

Sender

rdt_send(data)
sndpkt = make_pkt(data, checksum)
udt_send(sndpkt)

Wait for call from above

Wait for ACK or NAK

rdt_rcv(rcvpkt) && isNAK(rcvpkt)
udt_send(sndpkt)

Retransmission

rdt_rcv(rcvpkt) && isACK(rcvpkt)
Λ

Receiver

rdt_rcv(rcvpkt) &&
corrupt(rcvpkt)
udt_send(NAK)

Wait for call from below

rdt_rcv(rcvpkt) &&
notcorrupt(rcvpkt)
extract(rcvpkt,data)
deliver_data(data) ✓
udt_send(ACK)

In and Out of states?

Look at the "events"

69

rdt2.0 has a flaw!

Corrupted feedback: What happens if feedback ACK/NAK packet is corrupted?

- Sender doesn't know what happened at receiver! ACK or NAK?
- Scenario: Receiver transmits an ACK, but Sender receives NAK
  - Triggers Sender retransmission
    - → Receiver now has 2 copies of same file
    - Problem: Receiver doesn't know it's a Duplicate!!
- Q: What if the Receiver sends a NAK, but Sender receives an ACK?
  - What's the problem?

70

rdt1.0 → 2.0 → 2.1 → 2.2 → 3.0

71

rdt 2.1 The Fix

Channel Assumptions:
- Bit errors in both directions
- no packet loss

Goal:
- Detect corrupted ACKs, NAKs
- Detect duplicates

- Receiver:
  - NEW: Adds checksum to feedback packets (ACK/NAK)
  - Duplicate Detection: Examines the SN of packet received, if duplicate detected → discard
    - Keeps state about expected SN of received packets
- Sender:
  - NEW: Add Sequence Number (0 or 1) to each outgoing packet
    - Assists with Duplicate detection
  - If ACK/NAK "garbled" or NAK, Sender retransmits

What tools are we using now? (4 tools)

74

rdt2.1 Sender

What's new? SN = 0 or SN =1
Checksum: ACK/NAK valid?

rdt_send(data)
sndpkt = make_pkt(SN=0, checksum,data)
udt_send(sndpkt)

Wait for call from above use SN=0

Wait for ACK or NAK SN=0

rdt_rcv(rcvpkt) &&
(corrupt(rcvpkt) ||
isNAK(rcvpkt))
udt_send(sndpkt)

Retransmission

corrupt feedback or NAK

rdt_rcv(rcvpkt)
&& notcorrupt(rcvpkt)
&& isACK(rcvpkt)
Λ

move on

rdt_rcv(rcvpkt)
&& notcorrupt(rcvpkt)
&& isACK(rcvpkt)
Λ

move on...

Wait for ACK or NAK SN=1

Wait for call from above use SN=1

rdt_rcv(rcvpkt) &&
(corrupt(rcvpkt) ||
isNAK(rcvpkt))
udt_send(sndpkt)

retrans

rdt_send(data)
sndpkt = make_pkt(SN=1,checksum, data)
udt_send(sndpkt)

more data to send

75

rdt2.1 Receiver

What's new? Duplicate Detection!
--Check SN for Duplicates! Deliver only new data packets to APP Layer

Wait for SN0 from below

Wait for SN1 from below

1 rdt_rcv(rcvpkt) && (corrupt(rcvpkt)
sndpkt = make_pkt(NAK, chksum)
udt_send(sndpkt)

2 rdt_rcv(rcvpkt) &&
not corrupt(rcvpkt) &&
has_seq1(rcvpkt)
sndpkt = make_pkt(ACK, chksum)
udt_send(sndpkt)

drop duplicate
→ don't deliver!

3 rdt_rcv(rcvpkt) && notcorrupt(rcvpkt)
&& has_seq0(rcvpkt)
extract(rcvpkt,data)
deliver_data(data) ✓
sndpkt = make_pkt(ACK, chksum)
udt_send(sndpkt)

correct SN

rdt_rcv(rcvpkt) && (corrupt(rcvpkt)
sndpkt = make_pkt(NAK, chksum)
udt_send(sndpkt)

rdt_rcv(rcvpkt) &&
not corrupt(rcvpkt) &&
has_seq0(rcvpkt)
sndpkt = make_pkt(ACK, chksum)
udt_send(sndpkt)

drop duplicate

rdt_rcv(rcvpkt) && notcorrupt(rcvpkt)
&& has_seq1(rcvpkt)
extract(rcvpkt,data)
deliver_data(data) ✓
sndpkt = make_pkt(ACK, chksum)
udt_send(sndpkt)

correct SN

76

rdt2.1: Discussion

Sender:

- Seq # (0,1) added to pkt so Receiver can identify duplicates
- When does Sender retransmit?
  - Feedback:
    - NAK
    - Corrupted feedback

Receiver:

- Determine if received packet is Duplicate
  - Is SN the expected number?
    - Drop duplicates (App doesn't want them)
  - Rx sends ACK for all correctly received packets (regardless of SN)

One packet at a time!

77

rdt2.2

- Get rid of NAKs
  - Use ACKs only
- How?

78

rdt2.2: a NAK-free protocol

- Same functionality as rdt2.1, but get rid of NAKs (Use only ACKs)
- NEW: Receiver sends ACK# in its feedback
  - Instead of NAK, receiver sends SN (of last SN correctly received packet)
  - Note: Rdt2.1
    - ACK: indicates expected packet received and not corrupted
    - NAK: indicates received packet was corrupted
  - "duplicate ACK" → we will see this again!
- What tools does rtd2.2 use?

79

Rdt3.0 (last of the Rdt!!!!)

80

rdt3.0: Channel with errors *and* PACKET LOSS

New assumption: Underlying channel can drop packets (data or ACKs)

- Checksum, seq. #, ACKs, retransmissions don't solve this problem
- Why?

Approach: sender waits "reasonable" amount of time for feedback

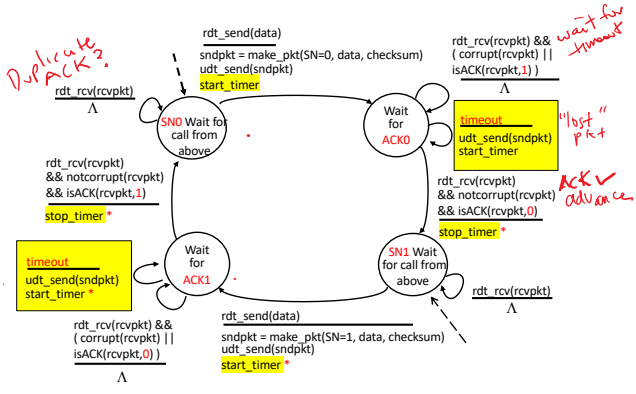
- If no ACK received in reasonable time → retransmission
- What If packet or ACK delayed (not lost)?
  - Retransmission will be duplicate, but we've handled this problem already ! (how?)
- Countdown timer required!*

81

Countdown Timer

82

rdt3.0 Sender –countdown timer for lost packet



Questions: 1. Circle all retransmissions
2. What does Sender do every time it retransmits?
2. When a corrupted ACK is received, does Sender retransmit?

84

What do you need to know about rdt?

- Ask yourself lots of "why" questions
- Understand goals of rdt protocol
- Understand the 4 primitives
- Iterations of new impairments on channel → identify problems and their solutions to fix the problem
- Look at a FSM and know follow action/transitions triggered by events
- For each rdt X:
  - what were the new channel impairments
    - what problem does that present?
    - how does the rdt version fix it
  - Fill in chart in Ch3 sample problems
- Do NOT memorize rdtX – it means understand for different impairments on the channel, what is the solution

86

5 versions of rdt protocol based on channel characteristics Practice!

Protocol version	Channel	Tools
Rdt 1.0	100% reliable	
Rdt 2.0	Bit errors (data)	
Rdt 2.1	Bit errors reverse path → duplicates	
Rdt 2.2	Get rid of NAKs	
Rdt 3.0	Packet Loss	

Tools:
- Checksum
- ACK
- NAK
- SN (sender)
- ACK #
- Timers

87